



UNIVERSITY OF
TORONTO

Engineering

AI/ML Bootcamp 2026

University of Toronto

Logistics

Time	Session	Location
9 am – noon	Morning session - 1-hour lecture - 2-hour practical	BF 323
Noon – 1 pm	Lunch	BF 315
1 pm – 4 pm	Afternoon session - 1-hour lecture - 2-hour practical	BF 323

- **Wi-fi access:** <https://eva.eis.utoronto.ca/>
- **GitHub access:** <https://github.com/uoft-ml-bootcamp/tutorials>
- **Seungjae Lee's email:** sjae.lee@utoronto.ca

Week 1

Session	Lecture (1hour)	Lab (2 hours)	Lecturer / TA
D1-M	• Introduction to AI and ML • Steps in ML projects 1	1. Python refresh 2. Visualization	S Lee / S Zhang, A Hooman
D1-A	• Steps in ML projects 2	1. Data preparation & feature selection 2. Model evaluation	S Lee / S Zhang, A Hooman
D2-M	• Supervised Learning 1 • Linear & Logistic regression	1. Linear Regression 2. Logistic Regression	S Lee / S Zhang, J Ko
D2-A	• Supervised Learning 2 • KNN • Decision trees	1. KNN 2. Decision Trees 3. Ensemble	S Lee / S Zhang, J Ko
D3-M	• Unsupervised Learning • K-means • PCA	1. PCA 2. K-means	S Lee / J Ko, A Hooman
D3-A	• ML Competition 1		S Lee / S Zhang, A Hooman
D4-M	• Company Visit – Themis Intelligence		CG Lee, S Lee
D4-A	• Deep Learning 1 • MLP	1. MLP	S Lee / J Ko, A Hooman
D5-M	• Deep Learning 2 • CNN • RNN	1. CNN 2. RNN	S Lee / S Zhang, J Ko
D5-A	• Evaluation 1		S Lee / S Zhang, J Ko

Week 2

Session	Lecture (1hour)	Lab (2 hours)	Lecturer / TA
D6-M	Deep learning 3 - GNN - PINN - Autoencoders	- GNN - PINN - Autoencoders	S Lee / J Ko, A Hooman
D6-A	Reinforcement learning 1 - MDP - Policy/Value iteration - Monte-Carlo	Frozen Lake	CG Lee / Koorosh
D7-M	Reinforcement learning 2	- Tabular RL - Deep RL	CG Lee / Ernesto
D7-A	Reinforcement learning 3	- Cart-pole - Policy optimization	CG Lee / Ernesto
D8-M	Transformer and LLMs 1	- Transformer - LLMs - Harnessing	Patel / Ramy
D8-A	ML Competition 2		Patel / Koorosh
D9-M	Transformer and LLMs 2	- RAG - Agentic AI	Patel / Jerry
D9-A	Lab Visit		Patel
D10-M	Evaluation 2		KH Park / Ramy
D10-A	No session		



UNIVERSITY OF
TORONTO

Engineering

Introduction to AI & ML



UNIVERSITY OF
TORONTO

Engineering

Machine Learning – Then & Now

History of AI
Turing Test
Driving Forces

History of AI



The Dawn of AI

1950



Perceptron

1958

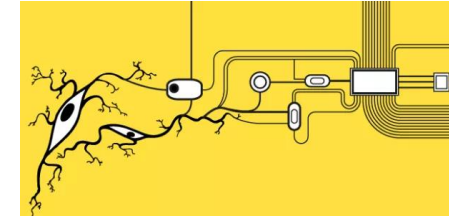
1956

The Dartmouth Conference



The Boltzmann Machine

1983



Deep Learning

2006



AlphaGo

2016

1997

IBM's Deep Blue



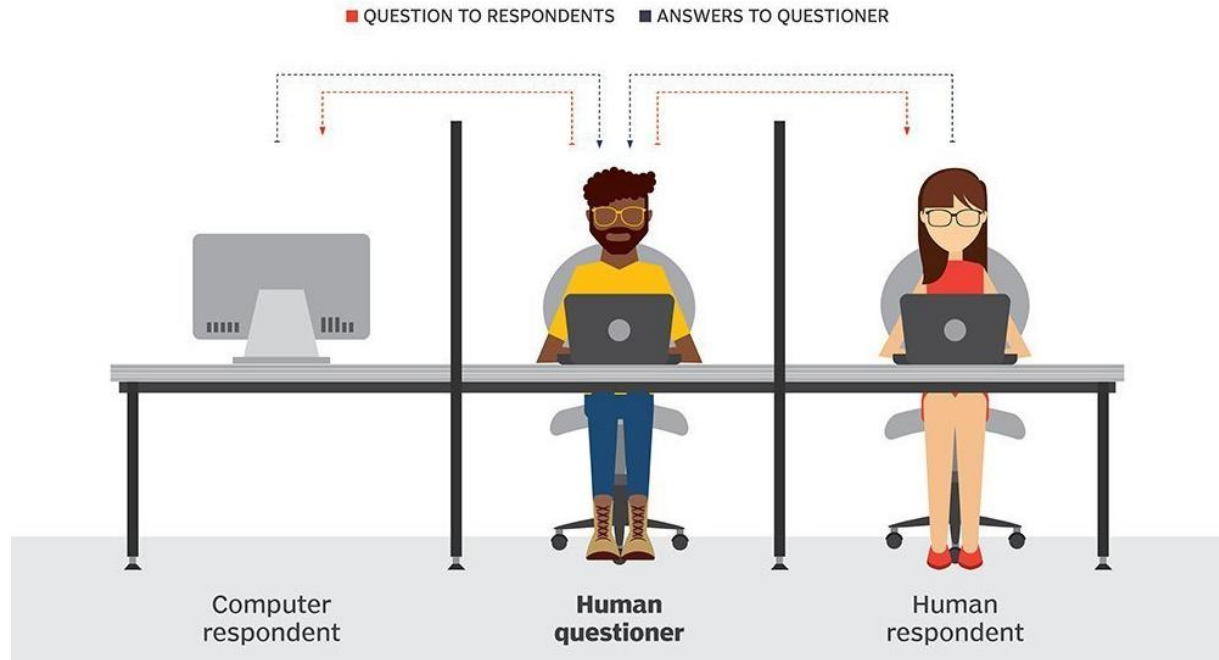
2011

IBM's Watson



Turing Test

During the Turing test, the human questioner asks a series of questions to both respondents.
After the specified time, the questioner tries to decide which terminal is operated by the human respondent and which terminal is operated by the computer.



Eugene Goostman

- First **pass** on Jun 6, 2014
- 13-year old boy from Odessa, Ukraine

Driving Forces

Big Data

More data was created this year than in last **5,000** years
(but only 0.5% was analyzed)

For example:



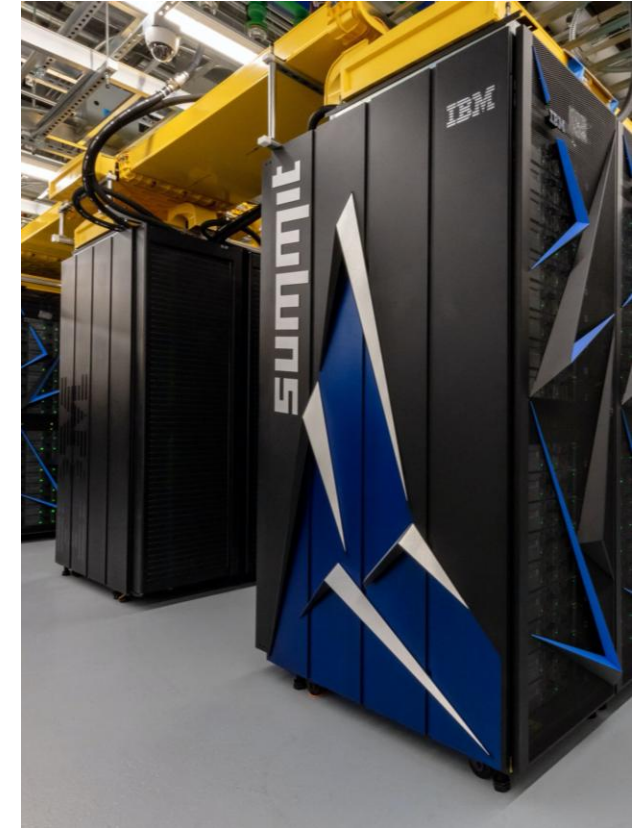
Driving Forces

Big Data

More data was created this year than in last **5,000** years
(but only 0.5% was analyzed)

Computing

If every person on Earth completes one calculation per second, it would take **305 days** to do what Summit can do in **1 second**





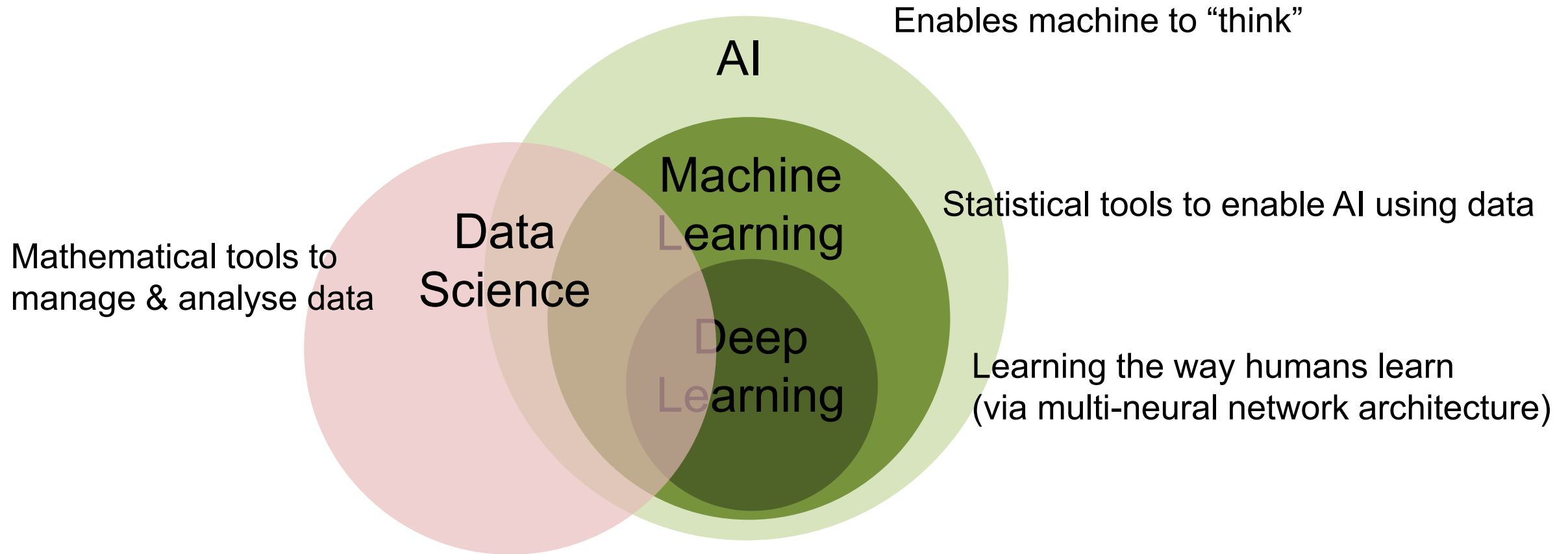
UNIVERSITY OF
TORONTO

Engineering

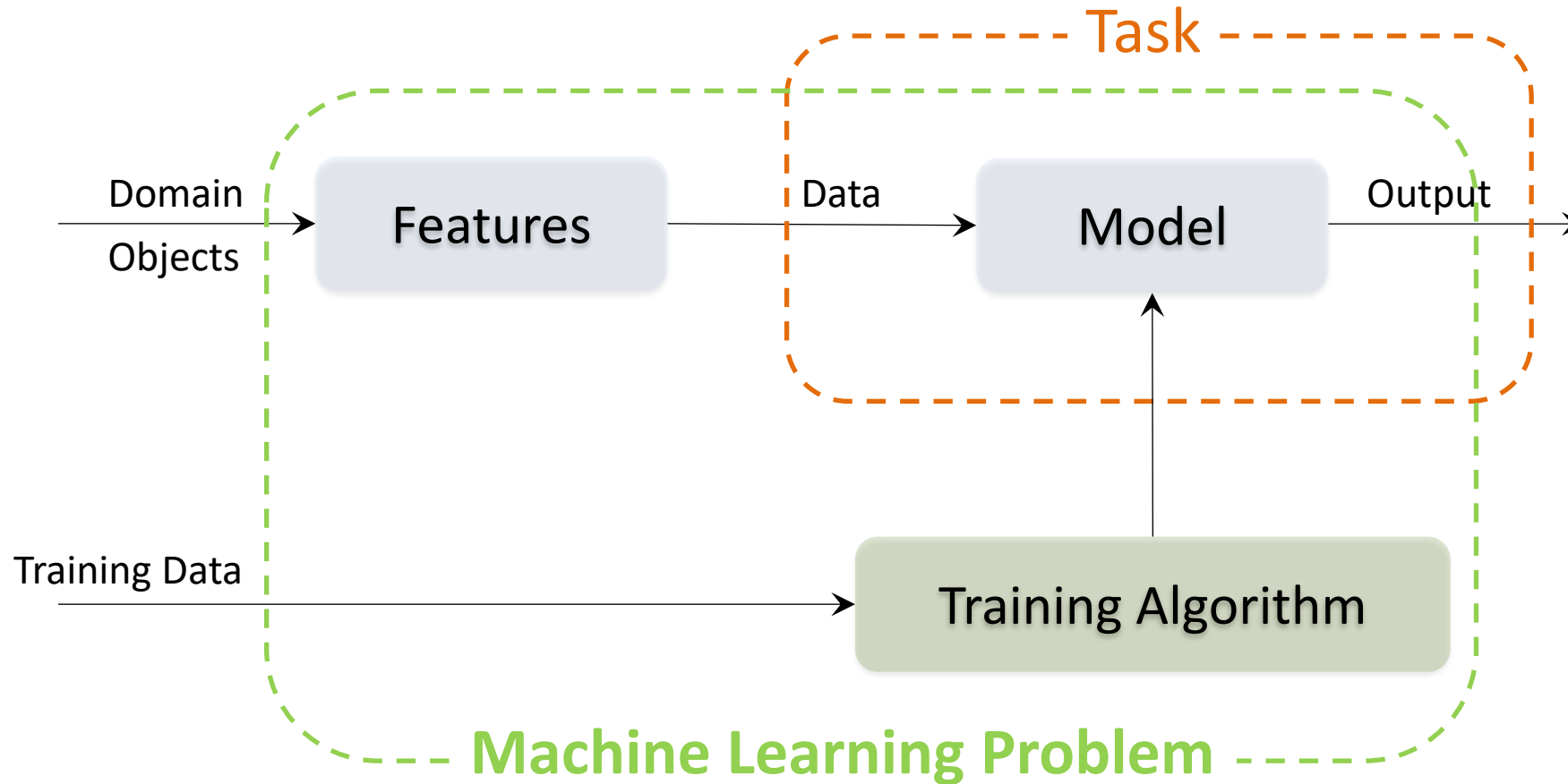
Basic Concepts of machine learning

Taxonomies of ML
Steps in ML Project

AI vs. ML vs...



Task, Training, Model and Data



Taxonomies of ML

Type of Learning/Supervision

- **Supervised ML**

- Learning by problems and solutions
- Using labeled examples to predict

X ₁	X ₂	X ₃	X _p	Y

Target

Supervised ML

- **Unsupervised ML**

- Learning by problems
- Explores data to learn hidden patterns

X ₁	X ₂	X ₃	X _p	Y

No
Target

Unsupervised ML

- **Reinforcement learning**

- Learning by trying
- Making strategic decisions

Taxonomies of ML

Type of Task

- **Regression**
 - Predict continuous value
- **Classification**
 - Predict discrete value
- **Decision making**
 - Predict the best alternative

X_1	X_2	X_3	X_p	Y
				5.2
				1.3
				23.0
				7.4

Numeric
Target

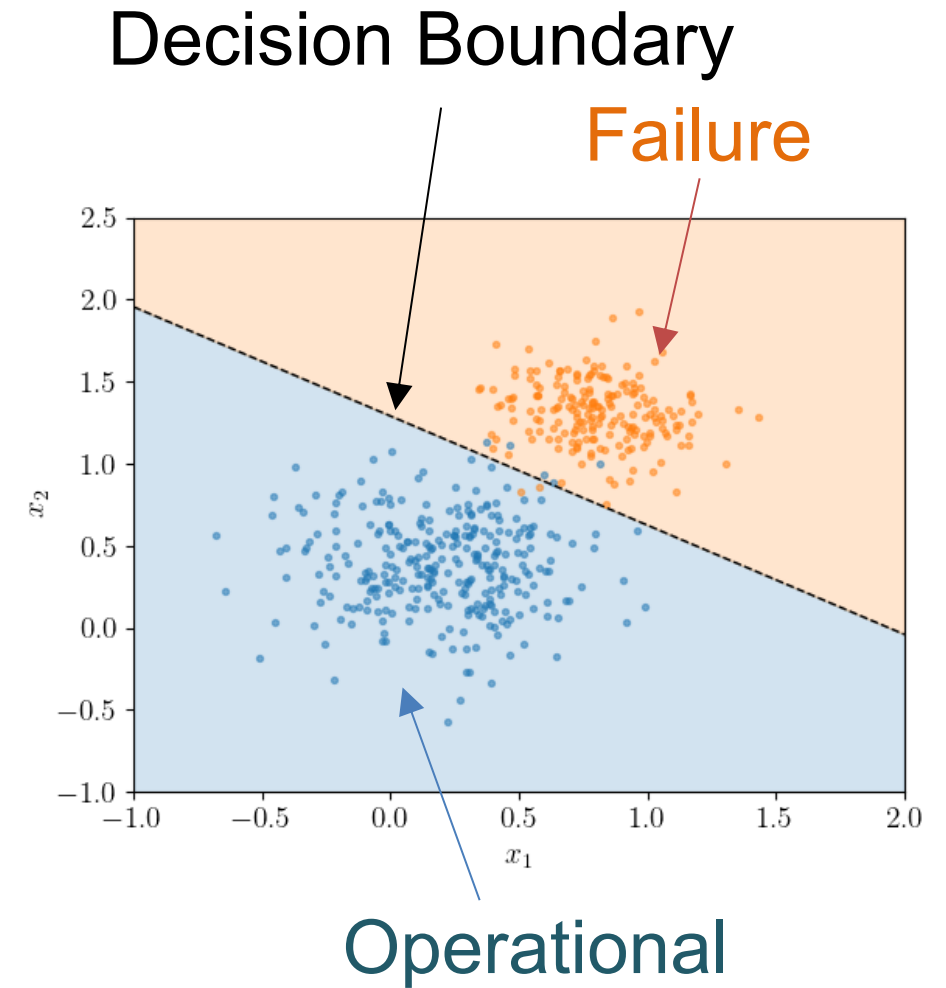
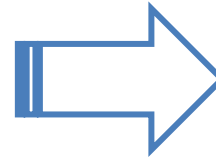
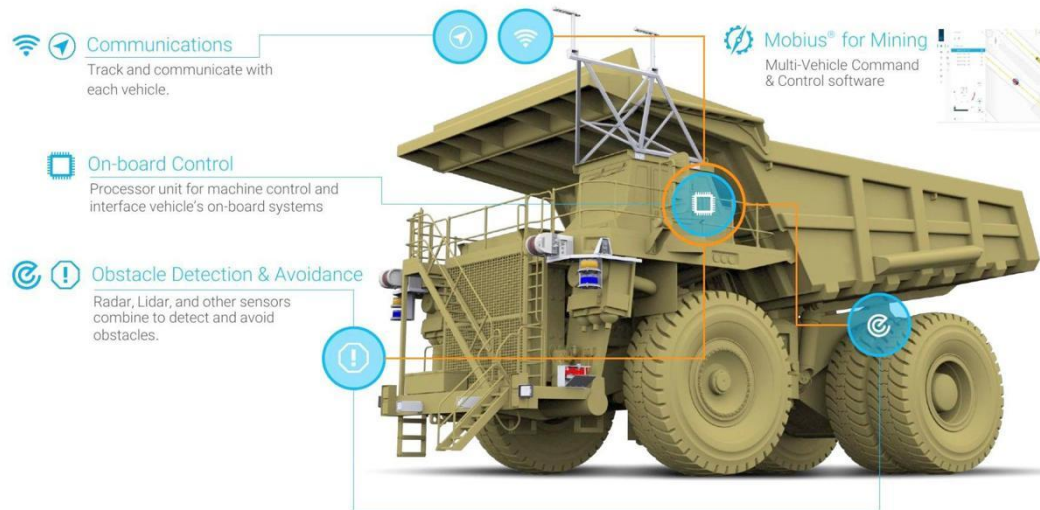
Regression Task

X_1	X_2	X_3	X_p	Y
				cat
				dog
				cat
				cat

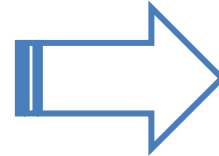
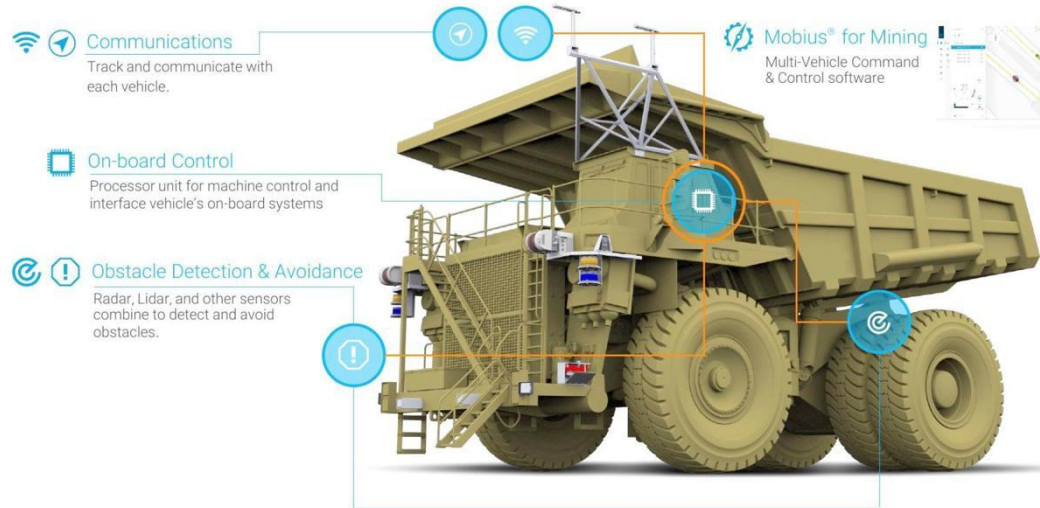
Categorical
"Labels"

Classification Task

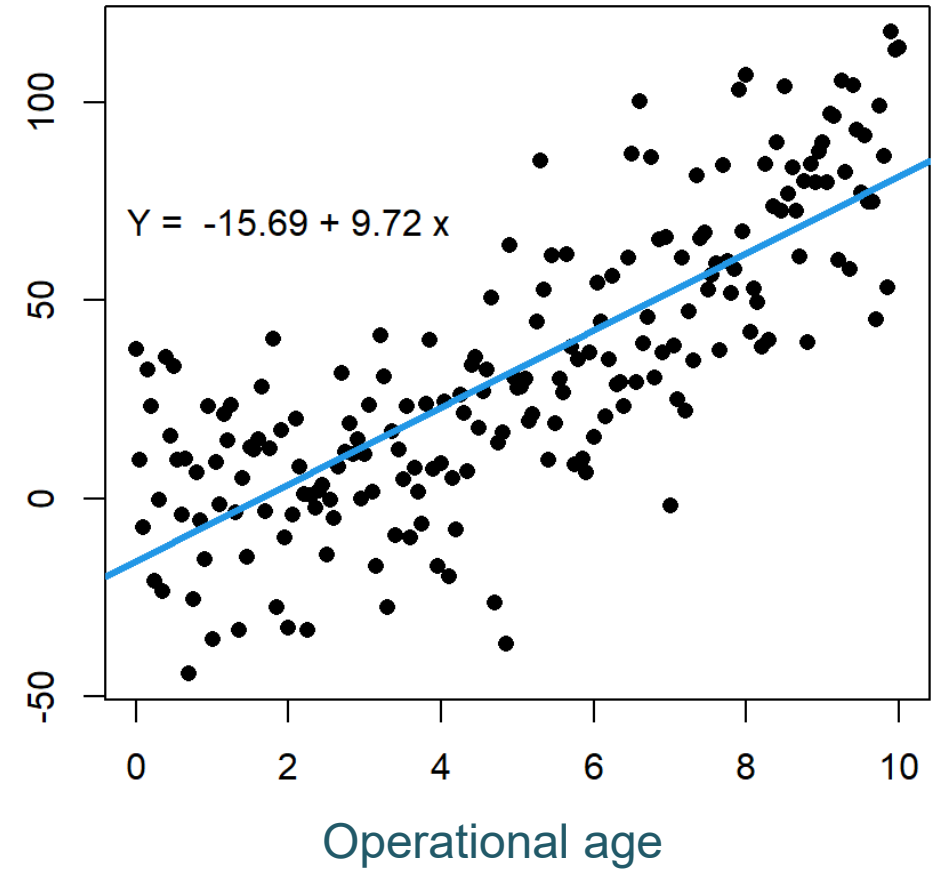
Classification Task



Regression Task

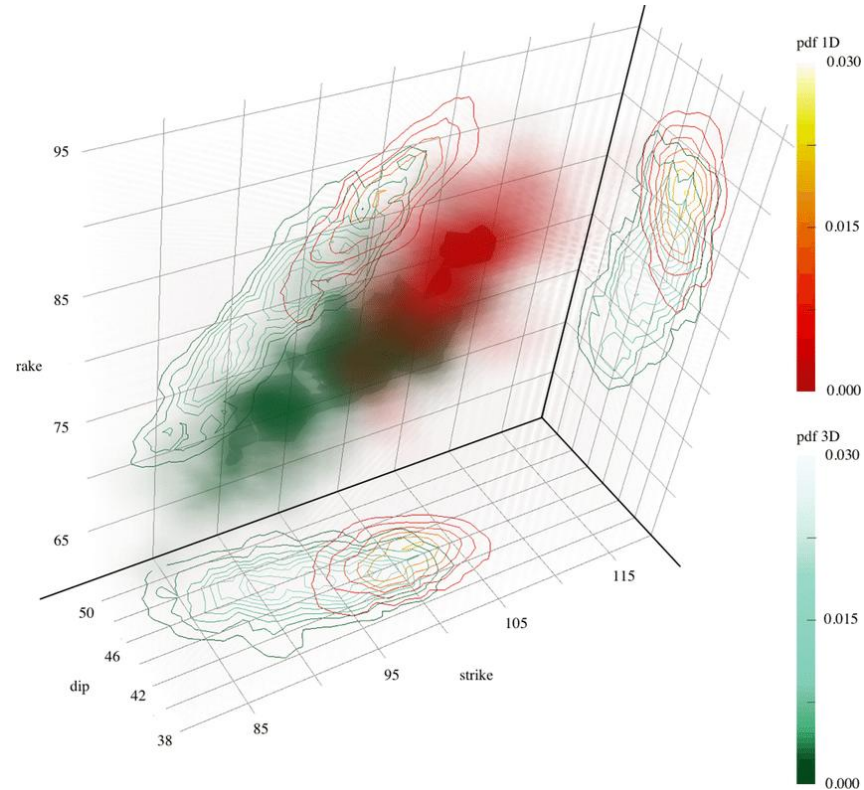


OM Cost



Task types

- Other tasks



dimension reduction



sequential mining

Steps in Machine Learning

1. Define the problem
2. Data Collection
The more, the better
3. Data Preparation
Duplicates, missing data, normalization, split
4. Choose a Model
Nature of tasks
5. Train the Model
Update learnable parameter to reduce loss
6. Evaluate the Model
Loss against unseen data
7. Tune Hyper-parameters of the Model
Learn unlearnables
8. Use the Model to Predict



UNIVERSITY OF
TORONTO

Engineering

Data Preparation & Performance Evaluation

Data Preparation
Performance Evaluation
A Case Study

Machine Learning in Practice

- Preparation of Data
 - One-hot Coding
 - Binning
 - Normalization
 - Standardization
 - Missing data & Imputation
 - Unbalanced data
- Three Sets
 - Training set
 - Validation set
 - Test set
- Performance Evaluation
 - Classification
 - Regression
- Selection of Model
 - Explainability
 - # of features and examples
 - Categorical vs. numerical features
 - Nonlinearity of the data
 - Training and Prediction speed
- Overfitting and Underfitting

Data collection

- Size
 - The more, the better
 - At least 10x more examples than number of trainable parameters
- Quality
 - Noise, even outliers
 - Missing values
 - Features
- Labeling
 - Direct vs. derived

Data Preparation

- **1-Hot Coding: Categorical to numerical**

Red = [1,0,0]		Red = 1
Yellow = [0,1,0]	VS.	Yellow = 2
Green = [0,0,1]		Green = 3

- **Binning: Numerical to categorical**

Age $\in [0, 5)$	$\rightarrow$	Bucket = 1
Age $\in [6, 12)$	$\rightarrow$	Bucket = 2
Age $\in [12, 20)$	$\rightarrow$	Bucket = 3

Data Preparation

- **Normalization: Range to [0,1]** – sensitive to outliers

$$\bar{x} = \frac{x - \min\{x_1, \dots, x_n\}}{\max\{x_1, \dots, x_n\} - \min\{x_1, \dots, x_n\}}$$

- **Standardization: Distribution to N(0, 1)** – robust to outliers

$$\hat{x} = \frac{x - \mu\{x_1, \dots, x_n\}}{\sigma\{x_1, \dots, x_n\}}$$

- **Why scale?**
 - Prevents large-range features from dominating gradient-based training and distance-based models
 - Many algorithms and their default settings are designed assuming features on similar scales

Data Preparation

- Missing Features and Data Imputation
 - Replacing with an average of the feature

$$\tilde{x} = \frac{1}{M} \sum_i^N x_i$$

	col1	col2	col3	col4	col5
0	2	5.0	3.0	6	NaN
1	9	NaN	9.0	0	7.0
2	19	17.0	NaN	9	NaN

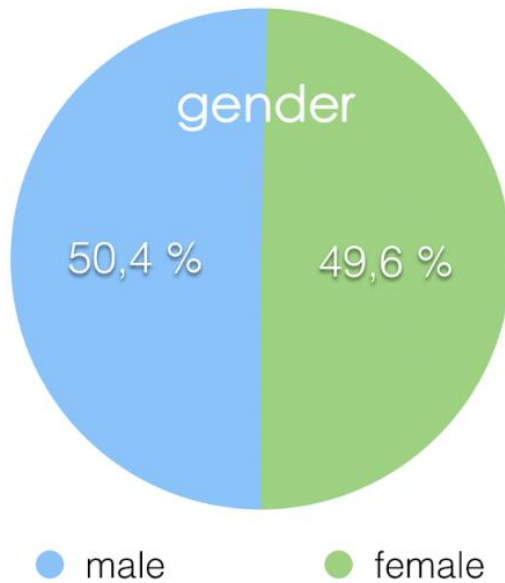
Data Preparation

- Other Data Imputation Methods
 - Most frequent value
 - 0 or any other constant
 - kNN
 - Regression
 - Extrapolation and interpolation

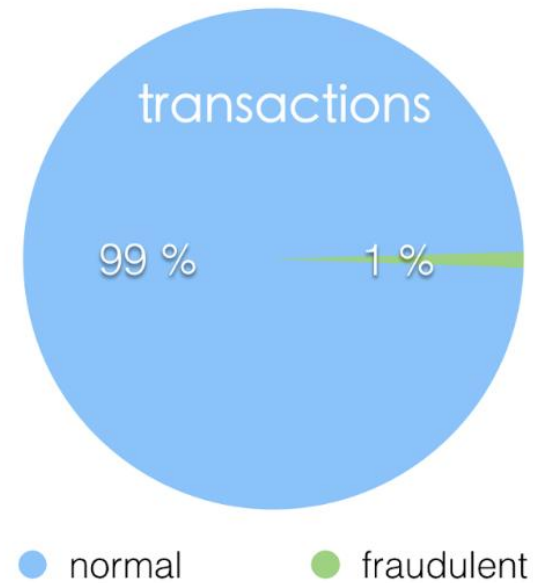
Data Preparation

- Unbalanced Datasets

Balanced Dataset



Unbalanced Dataset

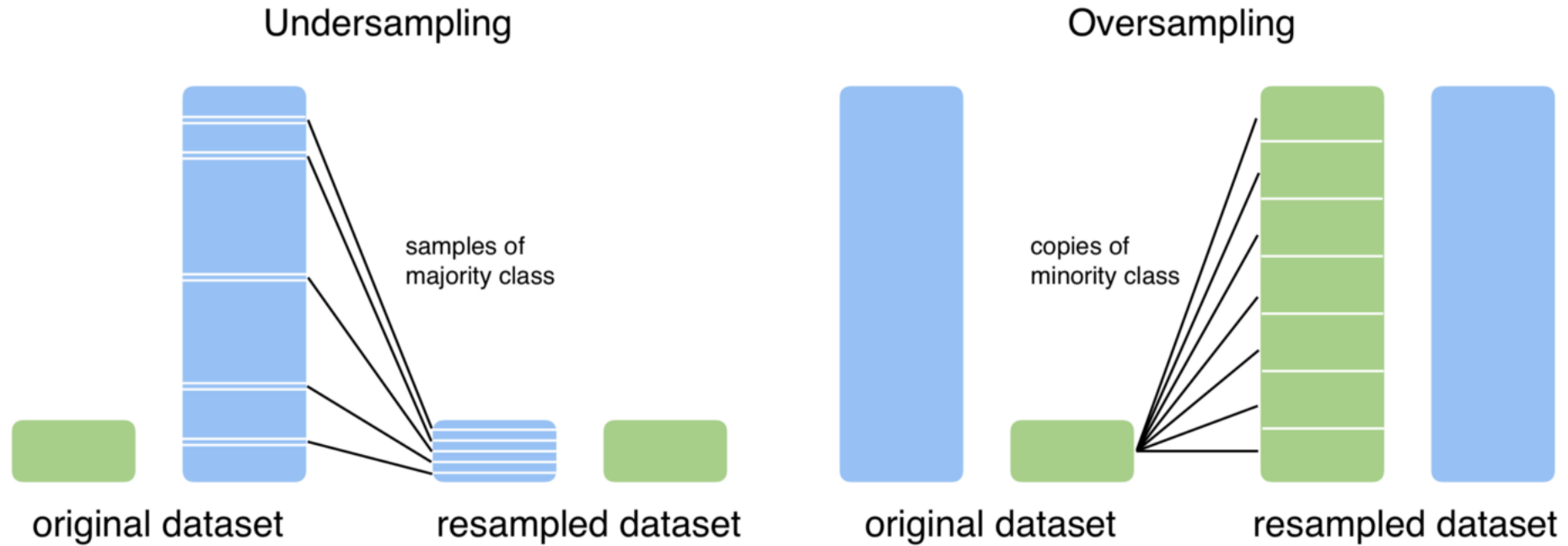


Data Preparation

- Unbalanced Datasets
 1. Carefully select performance metrics
e.g., Accuracy vs Precision+Recall
 2. Resampling
 1. Under-sampling
 2. Over-sampling
 3. SMOTE (Synthetic Minority Over-sampling Technique)

Data Preparation

- Unbalanced Datasets



Feature Engineering

- Creating new, more informative features from raw data
 - Better features often help more than a better model
- Transformations
 - $\log(x)$, ratios, differences → e.g., OM cost per operational year
- Date/time decomposition
 - Timestamp → hour, day-of-week, season
- Interaction features
 - Product of two features, e.g., speed $\times$ load
- Domain knowledge
 - e.g., temperature residuals after removing seasonality

Feature Selection

- Why not use all available features?
 - Curse of dimensionality: more features require many more examples (recall the 10x rule)
 - Irrelevant or redundant features add noise → overfitting
 - Slower training, harder to explain
- Goal
 - Keep the smallest subset of features that predicts well

Feature Selection: Three Approaches

Approach	How it works	Examples
Filter (fast)	Rank each feature before training, independent of the model	Correlation with target, variance threshold, mutual information
Wrapper (slow)	Search feature subsets by repeatedly training and scoring the model	Forward selection, backward elimination, RFE
Embedded (in between)	Selection happens automatically during model training	Lasso (L1), decision-tree feature importance

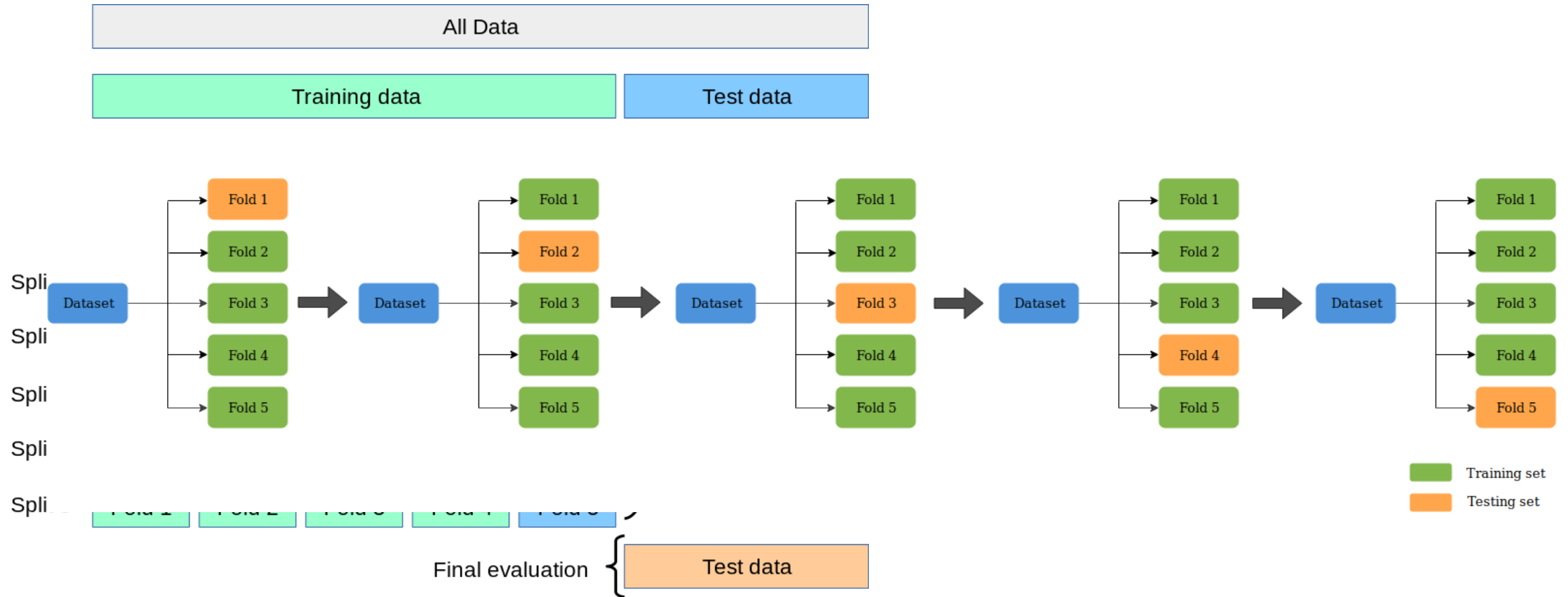
Three Sets

- Training Set
 - To build the model
- Validation Set
 - To tune hyper-parameters
- Test Set
 - To evaluate the final model on unseen data (used only once)
- Rule of Thumb
 - **70%:15%:15%** for **training : validation : test**

Cross Validation

- Cross validation (CV) usually means k-fold cross validation
 - Useful when data set is too small
 - In k-fold, every sample is used for training as well as for testing
 - Recommended in hyper-parameter tune up
- In k-fold CV
 - Data set is split into k subsets of equal size
 - The holdout method is repeated k times
 - Each time, $(k-1)$ sets are used for training and 1 set for testing

k-Fold Cross Validation



Data Leakage

- Information **unavailable at prediction time** leaks into training
 - The model “cheats” by seeing the answer, or part of it
- Symptom
 - Excellent validation/test scores, poor real-world performance
 - If results look too good to be true, suspect leakage first
- Example
 - Predicting machine failure using a sensor flag recorded after the failure occurred

Data Leakage

- Common sources
 - **Preprocessing before splitting**
 - Normalization/imputation fitted on all data → test-set statistics leak into training
 - **Target leakage**
 - A feature is derived from, or caused by, the label → e.g., “repair cost” to predict failure
 - **Duplicates**
 - Same rows appear in both training and test sets
 - **Temporal leakage**
 - Using future data to predict the past
 - **Group leakage**
 - Same patient / same pump appears in both training and test sets

Preventing Data Leakage

Situation	Wrong	Right
Scaling and imputation	Fit on all data, then split	Split first ; fit on training set, apply to validation/test
Cross validation	Preprocess once, outside CV	Refit preprocessing inside each fold (use a pipeline)
Time-series data	Random split	Split by time: train on past, test on future
Repeated objects (patient, pump)	Random split by row	Split by group
Every feature	Use anything correlated with the target	Ask: “Is it available at prediction time?”

Performance Evaluation

- Classification Tasks
 - Confusion matrix
 - Accuracy
 - Precision
 - Recall (sensitivity)
 - F1-score
 - AUC (Area Under ROC Curve)
- Regression Tasks
 - MAE
 - MSE
 - R^2

Classification Tasks

- Confusion Matrix

		Actual Value (as confirmed by experiment)	
		positives	negatives
Predicted Value (predicted by the test)	positives	TP True Positive	FP False Positive
	negatives	FN False Negative	TN True Negative

		Dove	Woodpecker	Robin	Crow	Starling	Sparrow	Finch	Goldfinch
Predicted	Dove	11	0	1	0	0	1	0	0
	Woodpe	0	9	0	1	1	0	0	0
	Robin	0	2	13	0	0	1	0	0
	Crow	1	0	0	9	0	0	0	0
	Starling	0	0	1	0	12	1	1	0
	Sparrow	0	1	2	0	0	13	1	0
	Finch	1	0	0	0	0	0	6	1
	Goldfinch	0	0	0	0	0	0	2	8

Evaluation of Binary Classification

– Accuracy

$$A = \frac{\text{\# of correct predictions}}{\text{\# of all predictions}}$$
$$= \frac{TP+TN}{TP+TN+FP+FN}$$

– Precision

$$P = \frac{\text{\# of correctly predicted positives}}{\text{\# of all predicted positives}}$$
$$= \frac{TP}{TP+FP}$$

– Recall (Sensitivity)

$$R = \frac{\text{\# of correctly predicted positives}}{\text{\# of actual positives}}$$
$$= \frac{TP}{TP+FN}$$

– F1 score

$$F1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$
$$= 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Evaluation of Binary Classification

	Actual Positive	Actual Negative	
Predicted Positive	TP=100	FP=10	$\hat{P}=110$
Predicted Negative	FN=5	TN=50	$\hat{N}=55$
	P=105	N=60	n=165

Accuracy

$$= \frac{100 + 50}{165} = 0.91$$

$$\text{Recall} = \frac{100}{100 + 5} = 0.95$$

$$\text{Precision} = \frac{100}{100 + 10} = 0.91$$

$$\text{F1 score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 0.93$$

Evaluation of Multi-Class Classification

		Dove	Woodpecker	Robin	Crow	Starling	Sparrow	Finch	Goldfinch	TP	FP	FN	Precision	Recall	F1 score
Predicted	Dove	11	0	1	0	0	1	0	0						
	Woodpe	0	9	0	1	1	0	0	0						
	Robin	0	2	13	0	0	1	0	0						
	Crow	1	0	0	9	0	0	0	0						
	Starling	0	0	1	0	12	1	1	0						
	Sparrow	0	1	2	0	0	13	1	0						
	Finch	1	0	0	0	0	0	6	1						
	Goldfinch	0	0	0	0	0	0	2	8						

Evaluation of Multi-Class Classification

		Dove	Woodpecker	Robin	Crow	Starling	Sparrow	Finch	Goldfinch	TP	FP	FN	Precision	Recall	F1 score
Predicted	Dove	11	0	1	0	0	1	0	0	11	2	2	0.846	0.846	0.846
	Woodpe	0	9	0	1	1	0	0	0	9	2	3	0.818	0.750	0.783
	Robin	0	2	13	0	0	1	0	0	13	3	4	0.813	0.765	0.788
	Crow	1	0	0	9	0	0	0	0	9	1	1	0.900	0.900	0.900
	Starling	0	0	1	0	12	1	1	0	12	3	1	0.800	0.923	0.857
	Sparrow	0	1	2	0	0	13	1	0	13	4	2	0.765	0.867	0.813
	Finch	1	0	0	0	0	0	6	1	6	2	4	0.750	0.600	0.667
	Goldfinch	0	0	0	0	0	0	2	8	8	2	1	0.800	0.889	0.842

Regression Task

- MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}|$$

- MSE (Mean Squared Error)

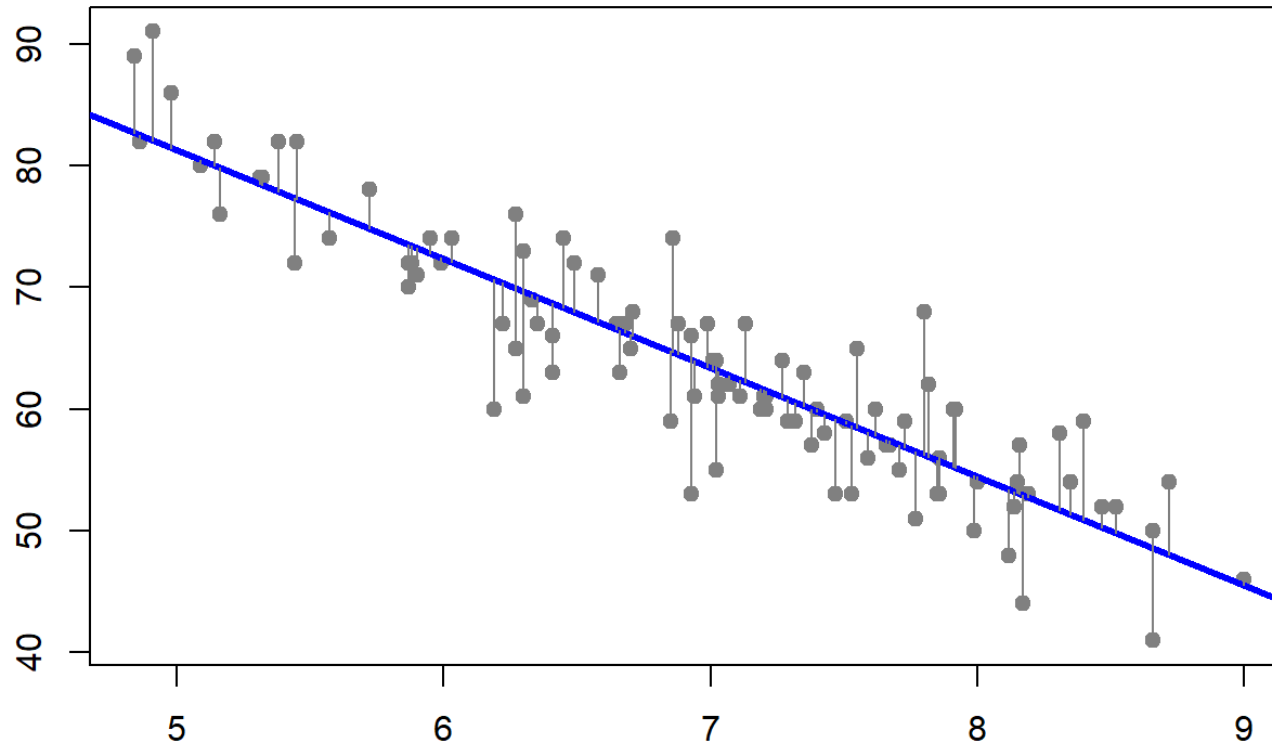
$$MSE = \frac{1}{n} \sum (y_i - \hat{y})^2$$

- R^2 (Coefficient of Determination)

$$R^2 = 1 - \frac{\text{Unexplainable Variation}}{\text{Total Variation}} = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

Regression Task

- MAE (Mean Absolute Error) and MSE (Mean Squared Error)

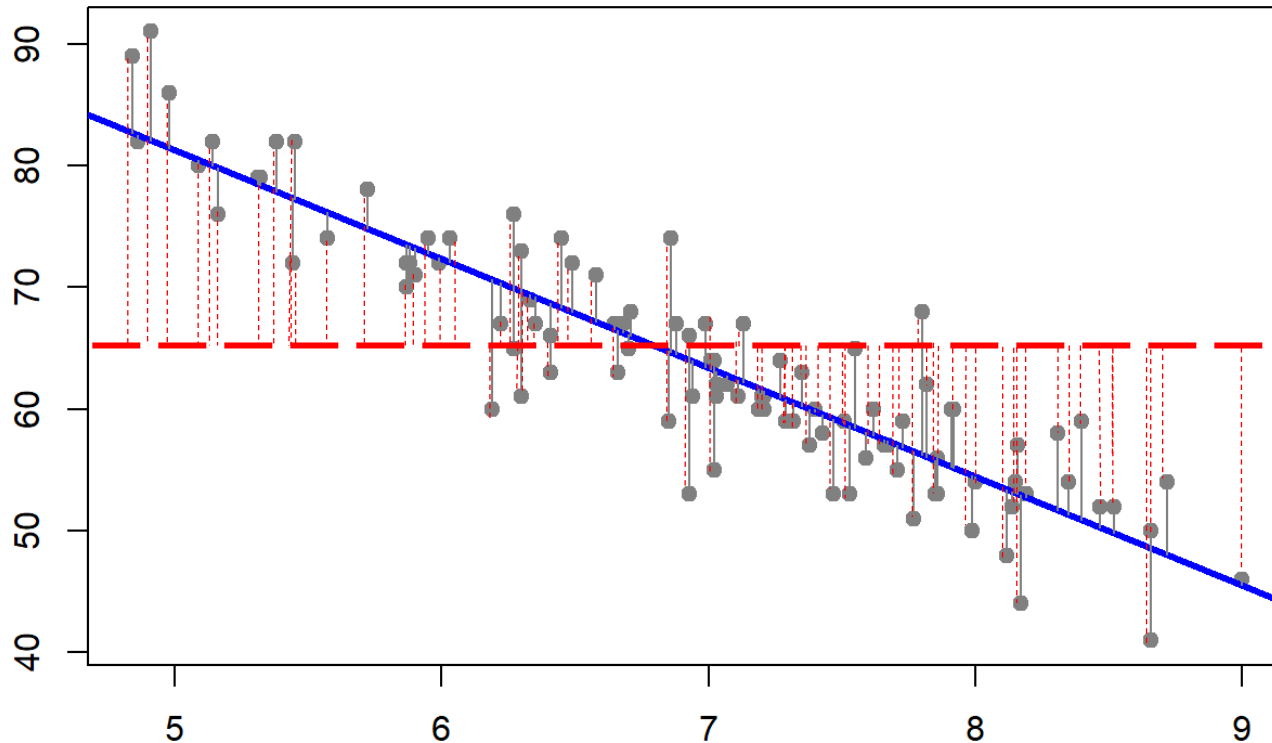


$$MAE = \frac{1}{n} \sum |y_i - \hat{y}|$$

$$MSE = \frac{1}{n} \sum (y_i - \hat{y})^2$$

Regression Task

- R^2



Sum of the squared residuals:

$$SS_{res} = \sum (y_i - \hat{y})^2$$

Total variability/variation:

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

Anomaly Detection on Track

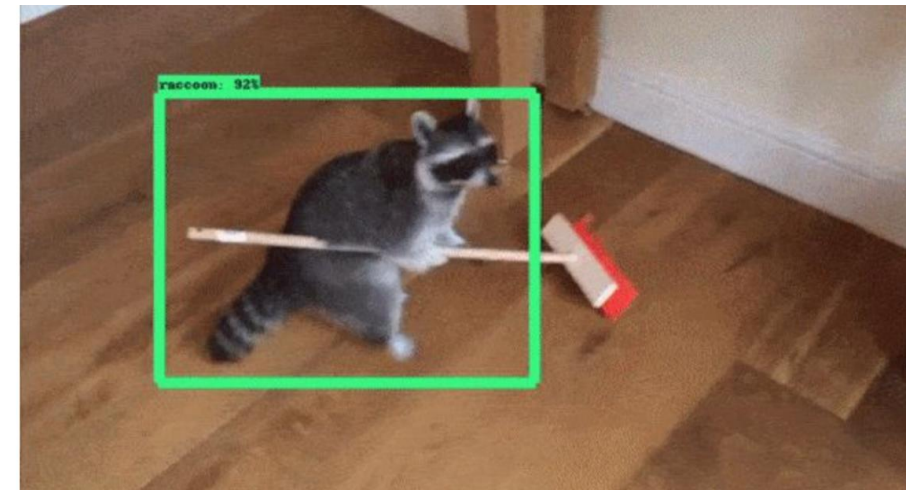
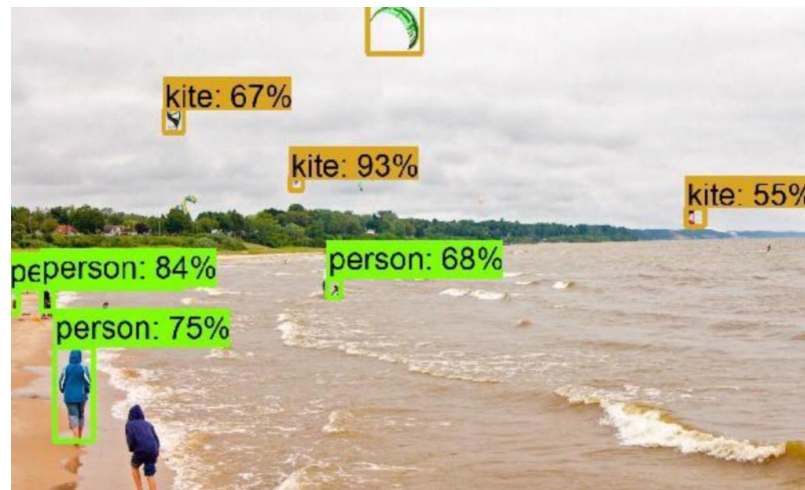


- Subway power-rail may overheat and cause defects
 - 6.5 Km light metro line
 - In May 2017, a train was damaged during operation
 - Power rail anomaly can be caused by high temperature
- IR camera is used to identify anomalies, but the video must be viewed manually to detect the anomalies

Anomaly Detection on Track



- Using TensorFlow Object Detection API & Microsoft Visual Object Tagging Tool to detect anomalies automatically

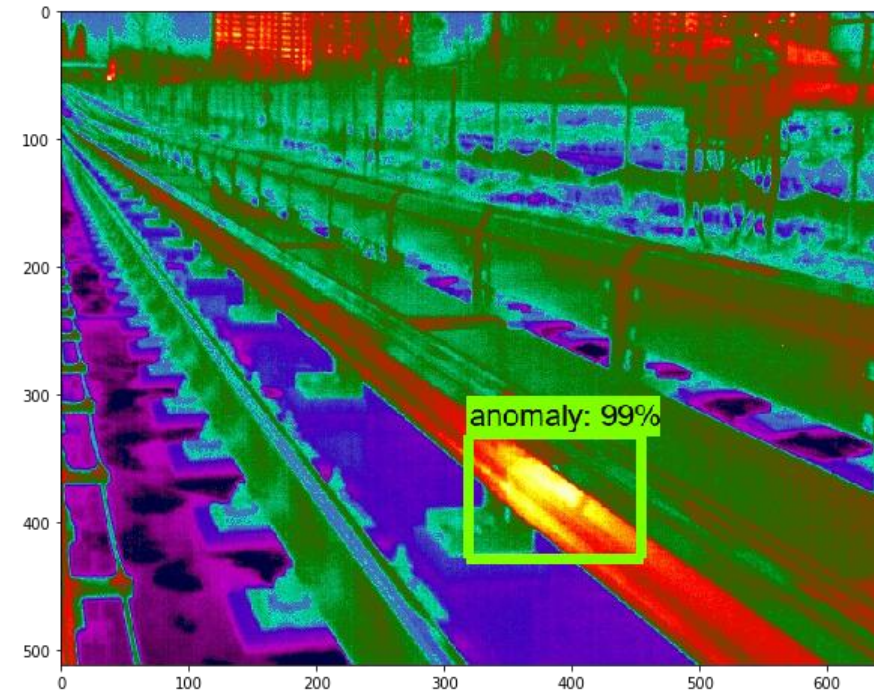
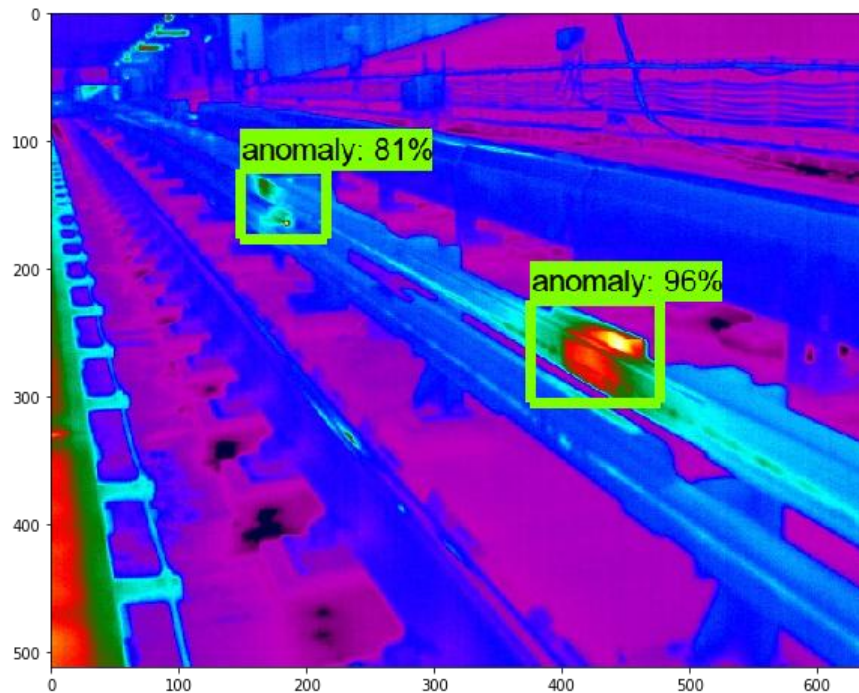


Data and Preparation

- 2 sets of videos
 - Line 3
 - 1 set has 4 videos with 16,000 frames recorded in December 2017
 - Only 3 videos capture the power rail
 - Partially labelled by TTC with a total 45 anomalies
 - Another set has 8 videos with 10,000 frames recorded in May 2018
 - Only 2 videos capture the power rail
 - Not labelled
 - Winter images were chosen
 - Recorded using a thermal camera in resolution 640X512
 - Further labelling was done by the team to get a total of 668 anomalies

Anomaly Detection on Track

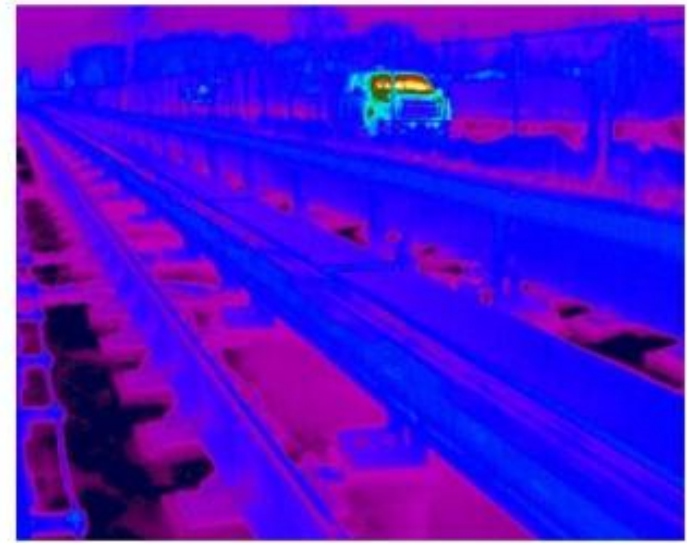
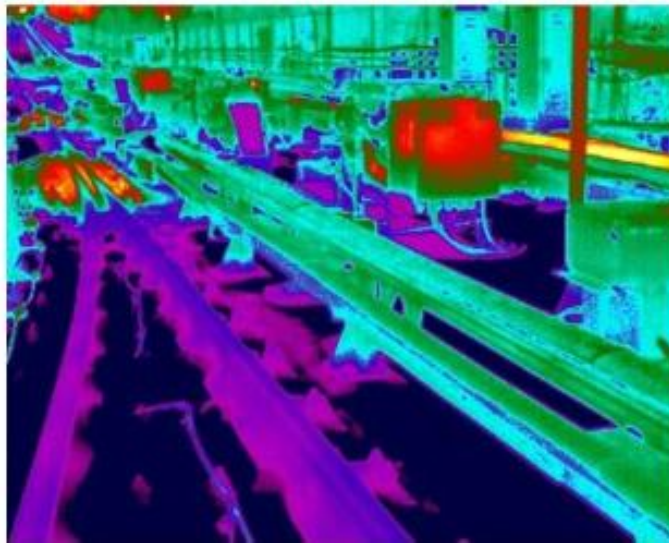
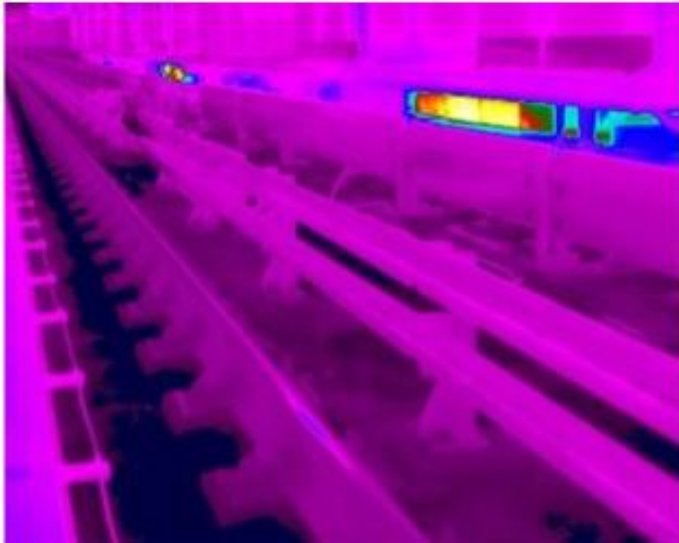
- ANN using Tensorflow Object Detection API



True Positives

Anomaly Detection on Track

- Tricky cases



Performance

- Balanced Tune-up

Method	Precision	Recall
Shallow AutoEncoder	47%	76%
Robust AutoEncoder	51%	88%
Isolation Forest	64%	94%
SVM	61%	93%

- Recall Maximization

Method	Precision	Recall
Isolation Forest	52%	100%
SVM	53%	100%

Selecting Model

- Explainability
- # of features and examples
- Categorical vs. numerical features
- Nonlinearity
- Training speed
- Prediction speed

ML Models

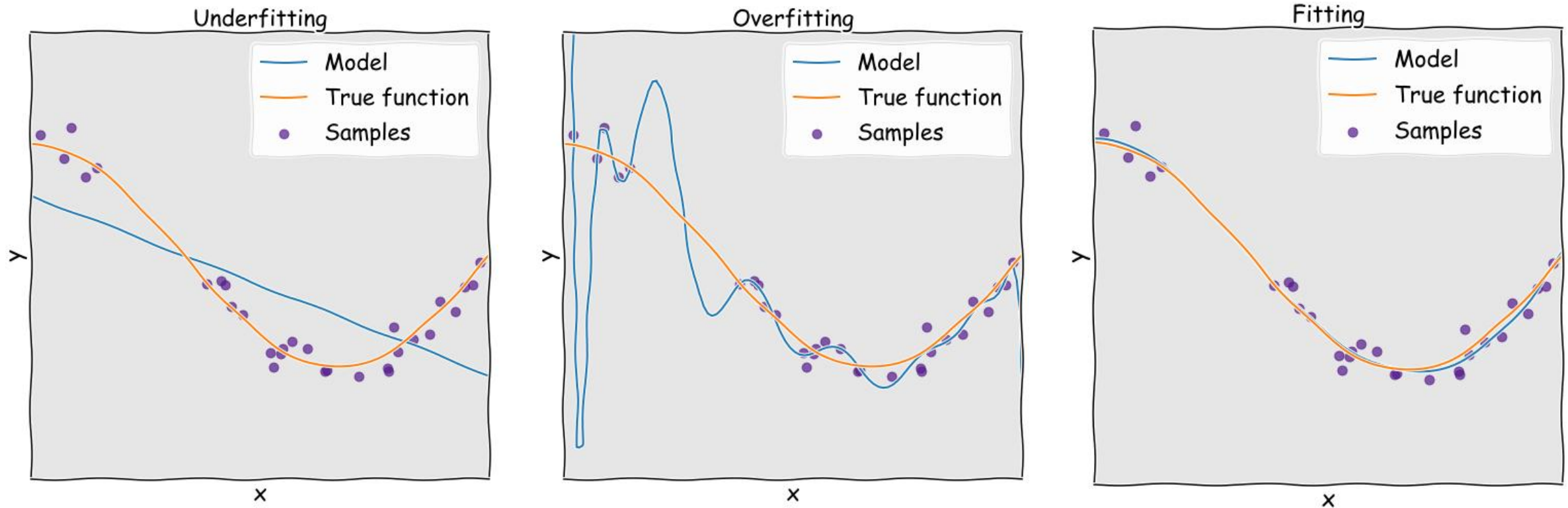
- Supervised ML
 - Linear Models
 - Linear Regression
 - Logistic Regression
 - Support Vector Machine
 - Non-linear Models
 - Naive Bayes Method
 - Decision Trees
 - Neural Networks
- Unsupervised ML
 - Principal Component Analysis
 - Clustering
 - Sequential Patterns

Recommendations for Model Selection

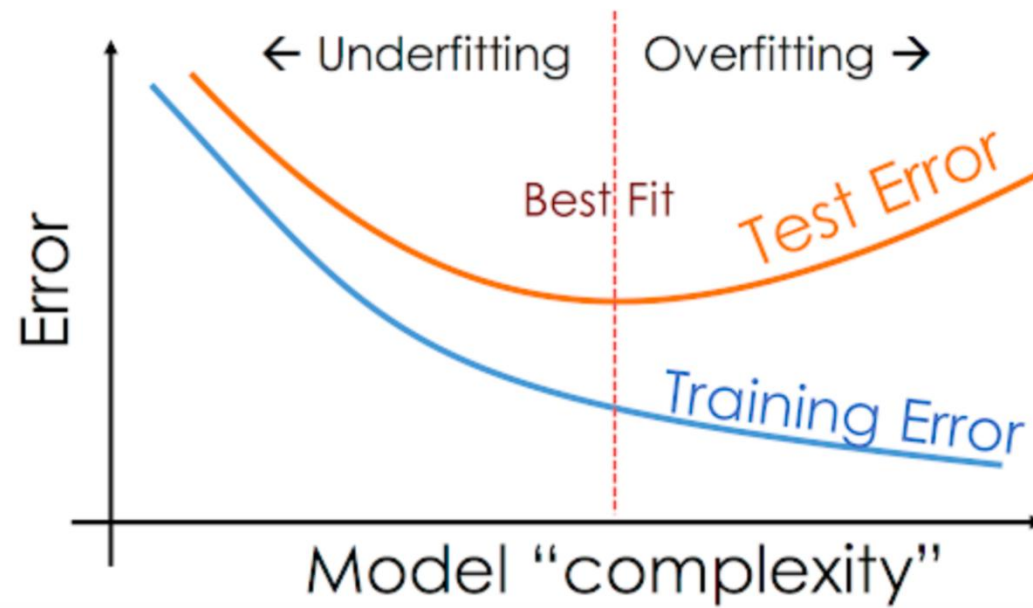
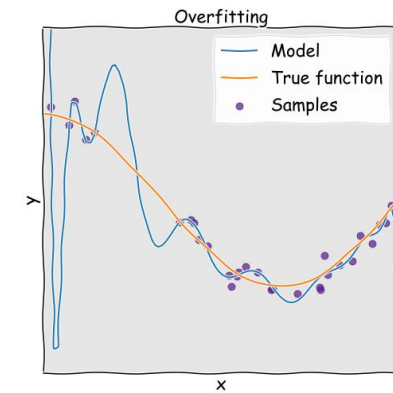
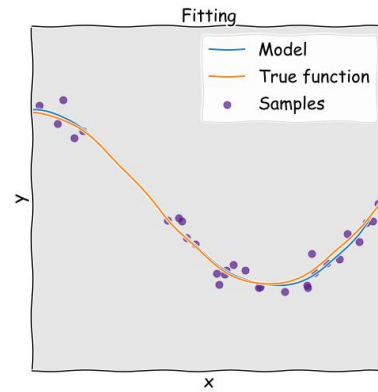
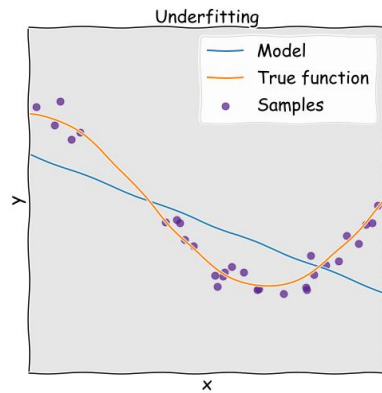
	Recommended	Not recommended
Explainability	kNN, linear regression, decision tree	NN, Kernel-methods
Large data set (feature/sample)	NN	SVM
Categorical	NN, Logistic regression, SVM, Decision tree	Linear regression
Non-linearity	NN, Kernel-methods	Linear regression, SVM
Training speed	Regression, Decision tree	NN
Prediction speed	Regression, NN	kNN, RNN

Model Complexity

- Under-fitting and Over-fitting



Overfitting and Underfitting



Occam's Razor

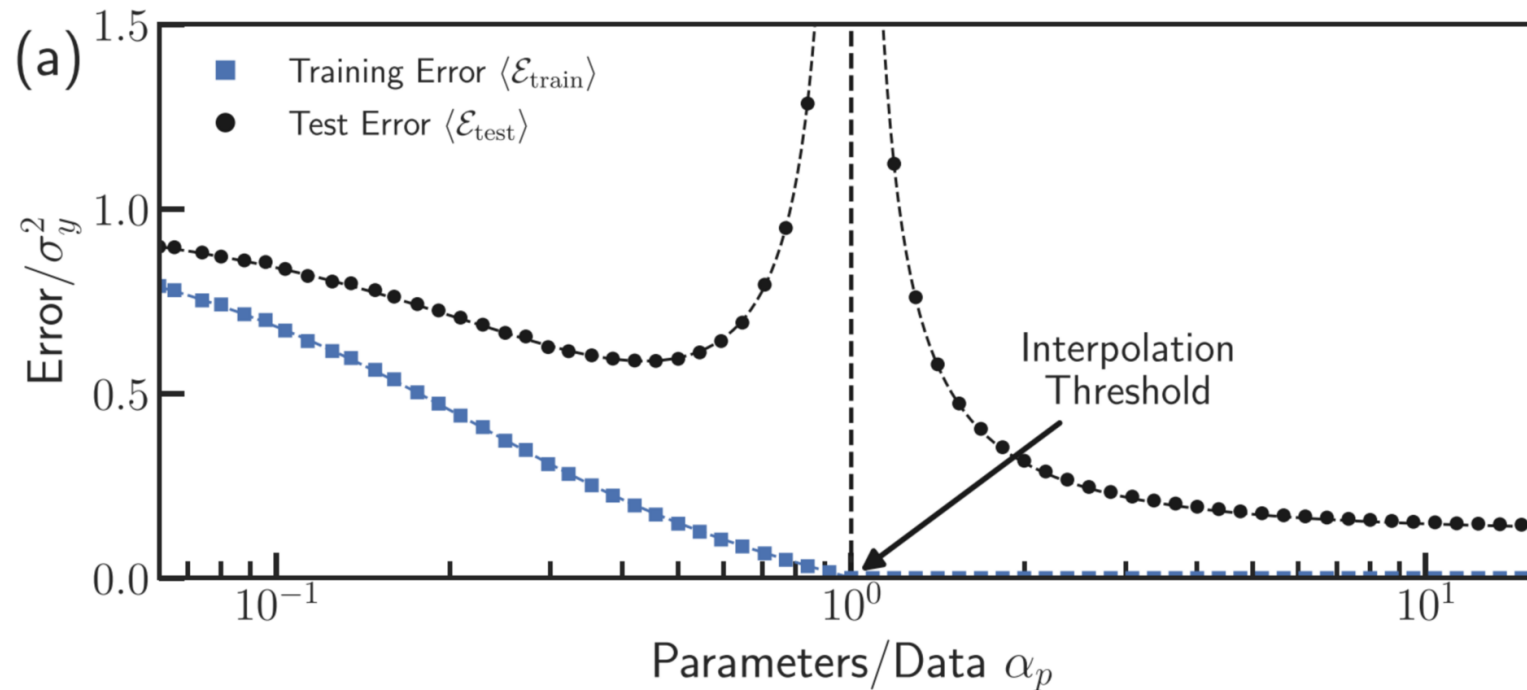
- *“Entities must not be multiplied beyond necessity”*
 - William of Ockham, 14th-century philosopher
 - Among explanations that work equally well, prefer the **simplest**
- In machine learning
 - Among models with **similar validation performance**, choose the simpler one
 - Simpler models generalize better, are easier to explain, and cheaper to run
- The razor in disguise, all week
 - Feature selection • Hyperparameter tuning • Regularization • Tree pruning
 - Note: the razor is a **tie-breaker**, not a law — a too-simple model just underfits

Overfitting and Underfitting

	Overfitting	Underfitting
Model Complexity	Higher	Lower
Degree of Training	More	Less
Size of Data	Smaller	Larger

Double Descent: A Modern Plot Twist

- Keep growing the model beyond the point where it fits the training data perfectly $\rightarrow$ test error can descend a second time (Belkin et al. 2019; Nakkiran et al. 2019)





UNIVERSITY OF
TORONTO

Engineering

Thank you!